to be performed (see chapter 8). This prevents the thread(s) populating the queue from running too far ahead of the thread(s) reading items from the queue.

The unbounded queue implementation shown here can easily be extended to limit the length of the queue by waiting on the condition variable in `push()`. Rather than waiting for the queue to have items (as is done in `pop()`), you need to wait for the queue to have fewer than the maximum number of items. Further discussion of bounded queues is outside the scope of this book; for now let's move beyond queues and on to more complex data structures.

6.3. Designing more complex lock-based data structures

Stacks and queues are simple: the interface is exceedingly limited, and they're very tightly focused on a specific purpose. Not all data structures are that simple; most data structures support a variety of operations. In principle, this can then lead to greater opportunities for concurrency, but it also makes the task of protecting the data that much harder because the multiple access patterns need to be taken into account. The precise nature of the various operations that can be performed is important when designing such data structures for concurrent access.

To see some of the issues involved, let's look at the design of a lookup table.

6.3.1. Writing a thread-safe lookup table using locks

A lookup table or dictionary associates values of one type (the key type) with values of either the same or a different type (the mapped type). In general, the intention behind such a structure is to allow code to query the data associated with a given key. In the C++ Standard Library, this facility is provided by the associative containers: `std::map<>`, `std::multimap<>`, `std::unordered_map<>`, and `std::unordered_multimap<>`.